

AI Development Ubuntu Desktop & LXD Container Setup Guide

PART 1 — Ubuntu AI Development Desktop

Tested on Ubuntu 22.04 LTS / 24.04 LTS. Run all commands as your normal user unless `sudo` is shown.

STEP 1 — System Update & Essential Base Tools

```
sudo apt update && sudo apt upgrade -y
```

```
sudo apt install -y \  
  build-essential \  
  curl \  
  wget \  
  git \  
  git-lfs \  
  unzip \  
  zip \  
  tar \  
  htop \  
  tree \  
  jq \  
  net-tools \  
  ca-certificates \  
  gnupg \  
  lsb-release \  
  software-properties-common \  
  apt-transport-https \  
  libssl-dev \  
  libffi-dev \  
  libxml2-dev \  

```

```
libxslt1-dev \  
zlib1g-dev \  
libbz2-dev \  
libreadline-dev \  
libsqlite3-dev \  
libncurses5-dev \  
libncursesw5-dev \  
xz-utils \  
tk-dev \  
liblzma-dev \  
libpq-dev \  
ffmpeg \  
libsm6 \  
libxext6 \  
libgl1-mesa-glx
```

STEP 2 — Python Version Manager (pyenv)

pyenv lets you install and switch between multiple Python versions cleanly.

```
# Install pyenv  
curl https://pyenv.run | bash  
  
# Add to ~/.bashrc (or ~/.zshrc if using zsh)  
echo '  
export PYENV_ROOT="$HOME/.pyenv"  
export PATH="$PYENV_ROOT/bin:$PATH"  
eval "$(pyenv init -)"  
eval "$(pyenv virtualenv-init -)"  
' >> ~/.bashrc  
  
source ~/.bashrc  
  
# Install latest stable Python (check pyenv install --list for latest)  
pyenv install 3.12.3  
pyenv global 3.12.3  
  
# Verify  
python --version  
pip --version
```

STEP 3 — Python Environment & Package Tools

```
# Upgrade pip, setuptools, wheel
pip install --upgrade pip setuptools wheel

# Install pipx (for isolated CLI tools)
pip install pipx
pipx ensurepath

# Install virtualenv and virtualenvwrapper
pip install virtualenv virtualenvwrapper

# Add virtualenvwrapper config to ~/.bashrc
echo '
export WORKON_HOME=$HOME/.virtualenvs
export VIRTUALENVWRAPPER_PYTHON=$(which python)
source $(which virtualenvwrapper.sh)
' >> ~/.bashrc

source ~/.bashrc
```

Miniconda (Alternative / Parallel to pyenv)

Use Miniconda if you prefer conda environments, especially for packages with complex C dependencies (e.g., CUDA, cuDNN).

```
wget https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86_64.sh
bash Miniconda3-latest-Linux-x86_64.sh -b -p $HOME/miniconda3

# Add to PATH
echo 'export PATH="$HOME/miniconda3/bin:$PATH"' >> ~/.bashrc
source ~/.bashrc

conda init bash
source ~/.bashrc

# Update conda
conda update -n base -c defaults conda

# Create a dedicated AI dev environment
conda create -n aidev python=3.12 -y
conda activate aidev
```

STEP 4 — Core Python Libraries for Data Science & AI

Install these inside your virtual/conda environment.

```
# Activate your environment first
# conda activate aidev (if using conda)
# workon aidev (if using virtualenvwrapper)

# — Core Scientific Stack —
pip install numpy scipy pandas matplotlib seaborn plotly

# — Machine Learning —
pip install scikit-learn xgboost lightgbm catboost

# — Deep Learning —
# PyTorch (CPU version – replace with CUDA version if you have a GPU)
pip install torch torchvision torchaudio --index-url https://download.pytorch.org/whl/cpu

# TensorFlow / Keras
pip install tensorflow

# — Transformer Models & NLP —
pip install transformers datasets tokenizers accelerate sentencepiece

# — LLM / AI API Libraries —
pip install openai anthropic langchain langchain-community langchain-core
pip install llama-index chromadb faiss-cpu

# — Vector Databases & Embeddings —
pip install pinecone-client weaviate-client qdrant-client

# — Data Processing & Feature Engineering —
pip install polars dask pyarrow fastparquet openpyxl xlrd xlwt

# — Jupyter & Notebooks —
pip install jupyterlab notebook ipywidgets ipykernel nbformat nbconvert
pip install jupyter-contrib-nbextensions

# Register kernel in Jupyter
python -m ipykernel install --user --name aidev --display-name "Python (aidev)"

# — MLOps & Experiment Tracking —
pip install mlflow wandb optuna hydra-core
```

```
# — Computer Vision —————  
pip install opencv-python-headless pillow imageio scikit-image  
  
# — Web & API —————  
pip install fastapi uvicorn flask requests httpx aiohttp pydantic  
  
# — Utilities —————  
pip install python-dotenv tqdm rich click typer loguru pytest black isort flake8 mypy  
pip install joblib pickle5 cloudpickle psutil
```

GPU Setup (NVIDIA — skip if CPU only)

```
# Check your CUDA version first  
nvidia-smi  
  
# Install CUDA Toolkit (example for CUDA 12.1)  
wget https://developer.download.nvidia.com/compute/cuda/repos/ubuntu2204/x86_64/cuda-keyring_1.1-1_all.deb  
sudo dpkg -i cuda-keyring_1.1-1_all.deb  
sudo apt update  
sudo apt install -y cuda-toolkit-12-1  
  
# Install cuDNN  
sudo apt install -y libcudnn8 libcudnn8-dev  
  
# Install PyTorch with CUDA support (replace cpu with cu121)  
pip install torch torchvision torchaudio --index-url https://download.pytorch.org/whl/cu121  
  
# Verify GPU is visible to PyTorch  
python -c "import torch; print(torch.cuda.is_available())"
```

STEP 5 — Development Tools

Git Configuration

```
git config --global user.name "Your Name"  
git config --global user.email "your@email.com"  
git config --global init.defaultBranch main  
git config --global core.editor "code --wait"  
  
# Generate SSH key for GitHub/GitLab
```

```
ssh-keygen -t ed25519 -C "your@email.com"
cat ~/.ssh/id_ed25519.pub # Copy this to GitHub/GitLab SSH keys
```

VS Code Extensions (since you already have VS Code)

Install these from the Extensions panel or via CLI:

```
code --install-extension ms-python.python
code --install-extension ms-python.vscode-pylance
code --install-extension ms-toolsai.jupyter
code --install-extension ms-toolsai.vscode-jupyter-cell-tags
code --install-extension ms-azuretools.vscode-docker
code --install-extension ms-vscode-remote.remote-containers
code --install-extension ms-vscode-remote.remote-ssh
code --install-extension GitHub.copilot # If you have a subscription
code --install-extension eamodio.gitlens
code --install-extension christian-kohler.path-intellisense
code --install-extension streetsidesoftware.code-spell-checker
code --install-extension mechatroner.rainbow-csv
code --install-extension ms-python.black-formatter
code --install-extension charliermarsh.ruff
```

VS Code **settings.json** Recommended Settings

Open with **Ctrl+Shift+P** → "Open User Settings (JSON)" and add:

```
{
  "python.defaultInterpreterPath": "~/.virtualenvs/aidev/bin/python",
  "editor.formatOnSave": true,
  "editor.rulers": [88],
  "[python]": {
    "editor.defaultFormatter": "ms-python.black-formatter",
    "editor.codeActionsOnSave": {
      "source.organizeImports": "explicit"
    }
  },
  "jupyter.notebookFileRoot": "${workspaceFolder}",
  "files.autoSave": "afterDelay"
}
```

JupyterLab Extensions

```
pip install jupyterlab-git
pip install jupyterlab-lsp python-lsp-server
pip install aquirdturtle-collapsible-headings
pip install jupyterlab_execute_time
```

STEP 6 — Docker

Docker is essential for containerizing your AI applications.

```
# Remove old versions
sudo apt remove docker docker-engine docker.io containerd runc 2>/dev/null

# Add Docker repo
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg
echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg] \
    https://download.docker.com/linux/ubuntu $(lsb_release -cs) stable" | \
    sudo tee /etc/apt/sources.list.d/docker.list

sudo apt update
sudo apt install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin

# Add your user to docker group (no sudo needed)
sudo usermod -aG docker $USER
newgrp docker

# Verify
docker run hello-world
docker compose version
```

STEP 7 — Database Tools

```
# PostgreSQL client + server
sudo apt install -y postgresql postgresql-contrib libpq-dev

# Redis
sudo apt install -y redis-server redis-tools
```

```
# SQLite browser
sudo apt install -y sqlitebrowser

# Python DB connectors
pip install psycpg2-binary redis pymongo sqlalchemy alembic

# DBeaver (universal GUI DB client)
wget https://dbeaver.io/files/dbeaver-ce_latest_amd64.deb
sudo dpkg -i dbeaver-ce_latest_amd64.deb
```

STEP 8 — LXD Setup (Host Side)

```
# Install LXD via snap (latest stable)
sudo snap install lxd --channel=latest/stable

# Add your user to the lxd group
sudo usermod -aG lxd $USER
newgrp lxd

# Initialize LXD
lxd init --auto

# Or use interactive setup for custom network/storage
lxd init

# Verify
lxc version
lxc list
```

STEP 9 — Additional Utilities

```
# Terminal multiplexer
sudo apt install -y tmux

# Better shell
sudo apt install -y zsh
sh -c "$(curl -fsSL https://raw.githubusercontent.com/ohmyzsh/ohmyzsh/master/tools/install.sh)"
```

```
# File manager
sudo apt install -y ranger

# System monitoring
pip install gpustat          # GPU monitoring
sudo apt install -y nvidia-smi # GPU top (NVIDIA)
sudo apt install -y bpytop

# API testing
sudo snap install postman

# Diagram / architecture docs
sudo snap install drawio

# Screen recording (for demo/tutorials)
sudo apt install -y obs-studio

# Ngrok (expose local services to internet for testing)
sudo snap install ngrok
```

PART 2 — LXD Sandbox Containers for Testing

*This covers what to install **inside** each container. You already have LXD running on the host.*

STEP 1 — Create & Configure a Base Container

```
# Launch Ubuntu 24.04 container
lxc launch ubuntu:24.04 ai-sandbox

# Enter the container
lxc exec ai-sandbox -- bash

# Inside the container — update system
apt update && apt upgrade -y
```

STEP 2 — Base Tools Inside the Container

```
# Run inside the container (lxc exec ai-sandbox -- bash)
apt install -y \
  build-essential \
  curl \
  wget \
  git \
  unzip \
  zip \
  tar \
  htop \
  tree \
  jq \
  net-tools \
  ca-certificates \
  gnupg \
  software-properties-common \
  libssl-dev \
  libffi-dev \
  libxml2-dev \
  libxslt1-dev \
  zlib1g-dev \
  libbz2-dev \
  libreadline-dev \
  libsqlite3-dev \
  libpq-dev \
  ffmpeg \
  libsm6 \
  libxext6
```

STEP 3 — Python Inside the Container

```
# Install Python 3.12 and pip
apt install -y python3.12 python3.12-venv python3.12-dev python3-pip

# Set python3.12 as default
update-alternatives --install /usr/bin/python python /usr/bin/python3.12 1
update-alternatives --install /usr/bin/pip pip /usr/bin/pip3 1

# Upgrade pip
```

```
pip install --upgrade pip setuptools wheel

# Create a virtual environment for isolation
python -m venv /opt/aienv
source /opt/aienv/bin/activate

# Make it activate on login
echo 'source /opt/aienv/bin/activate' >> ~/.bashrc
```

STEP 4 — Python Libraries Inside the Container

```
# Activate venv first
source /opt/aienv/bin/activate

# — Core Scientific Stack —
pip install numpy scipy pandas matplotlib seaborn plotly

# — Machine Learning —
pip install scikit-learn xgboost lightgbm

# — Deep Learning (CPU – containers rarely have GPU passthrough) —
pip install torch torchvision torchaudio --index-url https://download.pytorch.org/whl/cpu
pip install tensorflow-cpu

# — LLM & AI Libraries —
pip install openai anthropic langchain langchain-community transformers datasets

# — Data Processing —
pip install polars pyarrow fastparquet openpyxl

# — Web APIs (for testing your services) —
pip install fastapi uvicorn flask requests httpx pydantic

# — Utilities & Testing —
pip install pytest pytest-asyncio pytest-cov black isort flake8
pip install python-dotenv tqdm rich loguru joblib psutil

# — Jupyter (for interactive testing inside container) —
pip install jupyterlab notebook ipykernel
python -m ipykernel install --name aienv --display-name "Python (Container)"
```

STEP 5 — Database Clients Inside the Container

```
# PostgreSQL client only (not server – connect to host)
apt install -y postgresql-client

# Redis client
apt install -y redis-tools

# Python connectors
source /opt/aienv/bin/activate
pip install psycopg2-binary redis pymongo sqlalchemy
```

STEP 6 — Network & Port Access (Host Configuration)

Configure port forwarding so you can access services running inside the container from your host browser.

```
# On the HOST – get container IP
lxc list ai-sandbox

# Forward container port 8888 (Jupyter) to host port 8888
lxc config device add ai-sandbox jupyter-port proxy \
  listen=tcp:0.0.0.0:8888 \
  connect=tcp:127.0.0.1:8888

# Forward port 8000 (FastAPI/Flask)
lxc config device add ai-sandbox api-port proxy \
  listen=tcp:0.0.0.0:8000 \
  connect=tcp:127.0.0.1:8000

# Forward port 5432 (PostgreSQL) if running in container
lxc config device add ai-sandbox pg-port proxy \
  listen=tcp:127.0.0.1:5432 \
  connect=tcp:127.0.0.1:5432
```

STEP 7 — Shared Folder Between Host and Container

Mount your host project directory into the container for seamless code sharing.

```
# On the HOST
# Create a shared directory
mkdir -p ~/projects

# Add a disk device to the container
lxc config device add ai-sandbox projects disk \
  source=$HOME/projects \
  path=/home/ubuntu/projects

# Verify inside the container
lxc exec ai-sandbox -- ls /home/ubuntu/projects
```

STEP 8 — Create a Reusable Container Profile & Snapshot

```
# On the HOST

# Take a clean snapshot after setup is done
lxc snapshot ai-sandbox clean-setup

# To restore to this clean state anytime
lxc restore ai-sandbox clean-setup

# Create a profile from this container to spin up new ones quickly
lxc profile create ai-base
lxc profile copy default ai-base
# (or export container config to a profile manually)

# Export the container as an image for reuse
lxc publish ai-sandbox/clean-setup --alias ai-dev-base
# Later, launch new containers from this image:
lxc launch ai-dev-base new-container-name
```

STEP 9 — Start Services Inside the Container

Start JupyterLab (accessible from host browser)

```
# Inside the container
source /opt/aienv/bin/activate
```

```
jupyter lab --ip=0.0.0.0 --port=8888 --no-browser --NotebookApp.token='yourtoken'

# Access from host browser: http://localhost:8888
```

Start FastAPI app

```
# Inside the container
source /opt/aienv/bin/activate
uvicorn main:app --host 0.0.0.0 --port 8000 --reload

# Access from host browser: http://localhost:8000
```

Quick Reference: Container Management Commands

Task	Command (on Host)
List all containers	<code>lxc list</code>
Start container	<code>lxc start ai-sandbox</code>
Stop container	<code>lxc stop ai-sandbox</code>
Open shell in container	<code>lxc exec ai-sandbox -- bash</code>
Run one command	<code>lxc exec ai-sandbox -- python --version</code>
Copy file to container	<code>lxc file push myfile.py ai-sandbox/home/ubuntu/</code>
Copy file from container	<code>lxc file pull ai-sandbox/home/ubuntu/output.csv ./</code>
Take snapshot	<code>lxc snapshot ai-sandbox my-snapshot</code>
Restore snapshot	<code>lxc restore ai-sandbox my-snapshot</code>
Delete container	<code>lxc delete --force ai-sandbox</code>

Task	Command (on Host)
View container info	<code>lxc info ai-sandbox</code>
View resource usage	<code>lxc info --resources</code>

Python Library Reference Summary

Category	Libraries
Core Science	numpy, scipy, pandas, matplotlib, seaborn, plotly
ML	scikit-learn, xgboost, lightgbm, catboost, optuna
Deep Learning	torch, torchvision, tensorflow, keras
LLMs / NLP	transformers, datasets, langchain, openai, anthropic, llama-index
Vector DBs	faiss-cpu, chromadb, pinecone-client, qdrant-client
Data Processing	polars, dask, pyarrow, fastparquet, openpyxl
Web / API	fastapi, uvicorn, flask, requests, httpx, pydantic
MLOps	mlflow, wandb, accelerate, hydra-core
Notebooks	jupyterlab, notebook, ipywidgets, ipykernel
Dev Tools	black, isort, flake8, mypy, pytest, loguru, rich, python-dotenv
DB Connectors	sqlalchemy, psycopg2-binary, redis, pymongo, alembic
Computer Vision	opencv-python-headless, pillow, scikit-image